innovten

AI Agent Platform

Technical Design Reference

Purpose Enterprise-grade multi-agent orchestration service.
Audience Engineers joining the team. Read this before writing code.

Contents

1. What This Service Does

This service orchestrates AI agents that answer user questions and execute multi-step workflows. It sits between a frontend application and backend data stores (graph database, relational database), providing a structured, auditable, and budget-controlled AI layer.

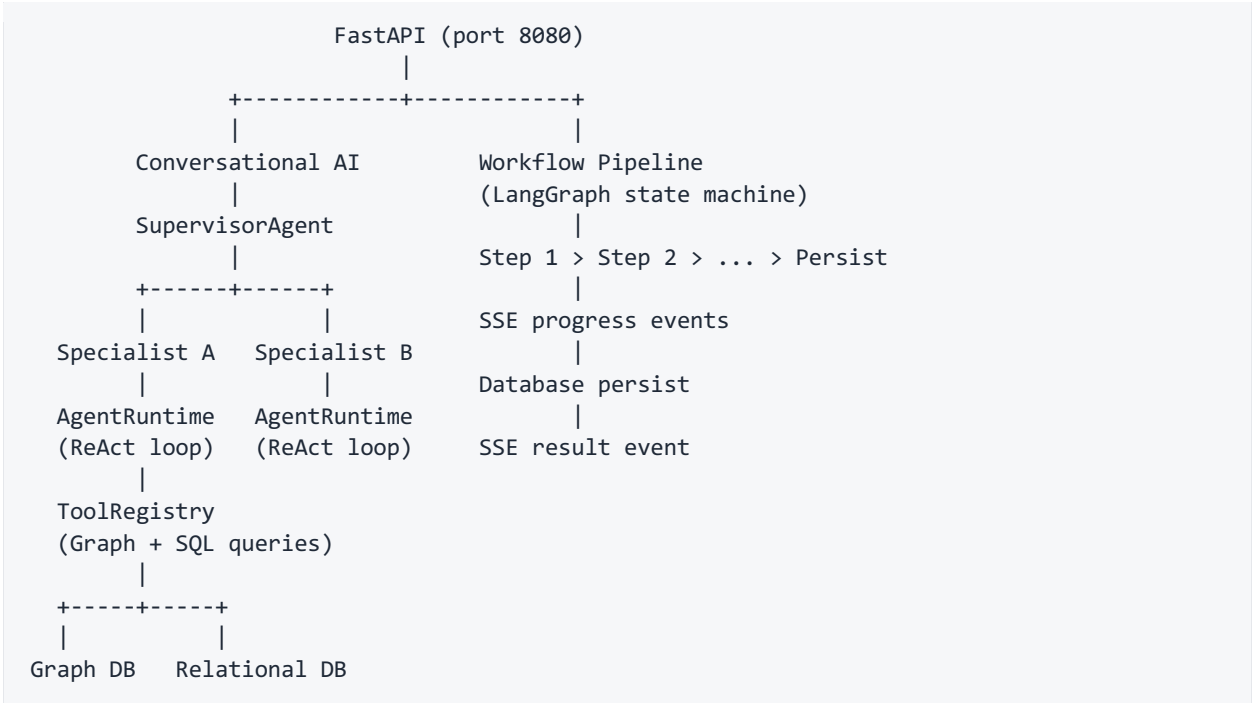
Two workloads:

1. **Conversational AI.** A supervisor routes user queries to specialist agents. Each specialist has access to specific tools (database queries, graph traversals). The agent gathers data, reasons over it, and returns a structured response the frontend renders.
2. **Workflow Pipelines.** A state machine executes multi-step analytical workflows (e.g., investigate an entity, draft a remediation plan). Each step emits real-time progress events via SSE. Results are persisted to the database.

Both workloads share infrastructure: LLM routing, safe database access, streaming, and YAML-driven configuration.

2. Architecture

Requests enter through FastAPI on port 8080 and fan out to one of two paths. The conversational path runs a supervisor that delegates to specialist agents, each backed by an AgentRuntime ReAct loop and a tool registry over the graph and relational databases. The workflow path runs a LangGraph state machine that executes ordered steps, streams progress over SSE, and persists results.



3. Design Principles

These are constraints, not suggestions. Every design decision traces back to one of these.

3.1 No Raw Queries

Every database interaction goes through an allowlist. No agent, tool, or runtime component can execute arbitrary SQL or graph queries.

Graph database. Query templates defined in YAML. Each template declares which agents may use it. A double-gate executor enforces: the agent must list the template AND the template must list the agent.

Relational database. Queries defined as named, parameterized objects in a query registry. A safe executor rejects any query name not in the agent's allowed list.

Rationale. LLM-generated tool calls are untrusted input. The allowlist ensures an LLM can only invoke pre-approved, parameterized queries. This eliminates injection risk at the architecture level, not at the validation level.

3.2 Budget Enforcement

Every LLM call has a cost. Every agent has a budget. Budgets are defined in configuration, not in code.

```
tool_calling:
  max_tool_iterations: 5
  max_llm_calls: 8
  max_cost_usd: 0.15
```

The runtime tracks calls, tokens, and estimated cost per request. When a budget is exceeded, the agent stops and returns what it has. It does not silently continue spending.

Rationale. Without budget enforcement, a single malformed query or adversarial prompt can trigger unbounded LLM spending. Budgets are the financial circuit breaker.

3.3 Tenant Isolation

Every request carries a tenant identifier in the auth header. This identifier flows through every layer:

- Stored in conversation sessions
- Injected into tool call parameters
- Passed to database queries as a filter
- Included in audit metadata

No tenant's data is visible to another tenant. If a tool call is missing the tenant parameter, the system auto-injects it from the request context.

3.4 Deterministic Over Probabilistic

Where possible, use deterministic logic instead of relying on the LLM.

Example: response type classification. Instead of asking the LLM to declare what type of response it produced, the system maps it deterministically from which tools were called:

```
TOOL_TO_RESPONSE_TYPE = {
    "fetch_entity_list": "entity_list",
    "fetch_entity_detail": "entity_detail",
    "fetch_relationship_graph": "graph_visualization",
}
```

Rationale. LLMs are good at selecting the right tool. They are unreliable at self-classifying their output. Use the LLM for what it is good at. Use code for what code is good at.

3.5 Fail Visible

Every error must be observable. No swallowed exceptions, no silent passes.

- **Tool failures:** logged with tool name, arguments, and error. Reported to the LLM as a tool result so it can adapt.
- **LLM failures:** logged with model, lane, and token count. Returned to the user as a structured error.
- **Pipeline failures:** error event emitted via SSE, then done event. Frontend shows the error.
- **Audit failures:** logged with retry count. Falls back to file logging if the database is down.

4. Multi-Agent Framework

4.1 Supervisor

The supervisor receives every user message. It decides what to do:

1. Build tool definitions from all registered specialists (OpenAI function-calling format)
2. Add a `respond_directly` tool for greetings and clarifications
3. Call the routing LLM (low temperature for consistency)
4. Parse the tool call: route to a specialist or respond directly
5. Apply output guardrails to the specialist's response

The supervisor never answers domain questions itself. It only routes. This separation ensures specialists can be added, removed, or modified without touching routing logic.

Circuit breaker. If a specialist fails, it is marked as broken with exponential backoff (30s, 60s, 120s, capped at 300s). During cooldown, the supervisor tells the user the specialist is temporarily unavailable instead of retrying and failing again.

4.2 Specialist Agents

Specialists are discovered at startup by scanning YAML configuration files. Each file defines:

- Which database queries the agent may use (allowlist)
- Which graph templates the agent may use (allowlist)

- LLM lane and temperature
- Tool iteration and cost budgets
- System prompt template path

If a specialist needs custom Python logic beyond the generic ReAct loop, register a custom class. Otherwise, the generic wrapper handles it.

Adding a new specialist requires zero code changes to the framework. Create a YAML file, create a prompt template, restart. The registry discovers it.

4.3 ReAct Loop (AgentRuntime)

The runtime is the execution engine for every specialist. It implements a tool-calling loop:

```
Iteration 0: Call LLM (force tool usage) -> execute tool calls
Iteration 1: Call LLM (force tool usage) -> execute tool calls
Iteration 2: Call LLM (auto) -> tool calls or final response
...
Final: Parse LLM output into structured response
```

Key behaviors

- **First N iterations force tool usage.** The LLM must gather data before answering. This prevents hallucinated responses that skip the database entirely.
- **Tool results are truncated** to a configurable character limit to protect the context window.
- **Follow-up suggestions are validated** against available tool capabilities. Questions the agent cannot answer are filtered out.
- **Response type is set deterministically** from which tools were called, not from LLM self-classification.

4.4 Structured Response

Every agent returns a canonical response envelope:

```
{
  "agent": "specialist_name",
  "responseType": "entity_list",
  "status": "success",
  "blocks": [
    {"type": "text", "content": {"body": "Analysis text..."}},
    {"type": "table", "content": {"title": "...", "columns": [...], "rows": [...]}},
    {"type": "graph", "content": {"nodes": [...], "edges": [...]}}
  ],
  "payload": { "structured data specific to responseType" },
  "thinking": ["Step 1...", "Step 2...", "Found 5 results..."],
  "followUps": ["Suggested question 1?", "Suggested question 2?"],
  "sources": [{"tool": "fetch_entity_list", "recordCount": 5}],
  "meta": {"tokensUsed": 8200, "costUsd": 0.03, "durationMs": 16300}
}
```

The frontend uses `responseType` to select a rendering template and `blocks` for content. The `meta` field is for debugging and cost tracking, not displayed to users.

4.5 Guardrails

Input guardrails (applied before the supervisor):

- Message length limits
- Unicode normalization (defeats homoglyph injection attacks)
- Zero-width character stripping
- Pattern-based prompt injection detection

Output guardrails (applied after the specialist):

- Strip raw query fragments from text blocks
- Redact PII patterns (emails, phone numbers)
- Enforce maximum block count per response
- Payload sanitization: recursive PII redaction, size limit, depth limit

Guardrails are not optional. They run on every request and every response.

5. Workflow Pipelines

5.1 State Machine

Workflows are implemented as LangGraph directed graphs. Each node:

- Receives shared mutable state
- Makes LLM calls and/or database queries
- Updates state with results
- Emits SSE step events for real-time progress

Nodes are isolated. A failure in one node does not corrupt state from previous nodes. Budget tracking is per-node so expensive steps can have tighter limits.

5.2 Persistence

The final node persists all results in a single database transaction. If any statement fails, the entire transaction rolls back. Partial writes never reach the database.

5.3 Real-Time Streaming

All progress communication uses Redis pub/sub:

```
Publisher (backend node) -> Redis channel -> Subscriber (SSE endpoint) -> Frontend
```

The backend task runs independently of the SSE connection (`asyncio.create_task`). If the frontend disconnects, the workflow continues and persists results. The user can navigate away and find the completed result later.

6. Conversation System

6.1 Dual-Write Persistence

Every conversation turn writes to both:

- **Cache (Redis):** session metadata and recent messages. TTL-based expiry. Fast reads for subsequent turns.
- **Durable store (Postgres):** full transcript, usage tracking, conversation status.

On load, the cache is checked first. If expired, the session is restored from the durable store. This gives sub-millisecond reads for active conversations without risking data loss.

6.2 Usage Ceiling

Each conversation has configurable limits (max messages, max tokens). At 80%, a warning is emitted. At 100%, the conversation is locked and the user is prompted to start a new one. This prevents unbounded cost accumulation on long-running conversations.

6.3 Context Injection

The conversation agent receives page context from the frontend (what entity the user is looking at). This context:

- Determines which prompt template to use
- Pre-fetches relevant data before the LLM sees the question
- Filters tool access to relevant queries

This means the same agent gives different answers on different pages because it has different context, not because it is a different agent.

7. LLM Routing

7.1 Lane-Based Routing

LLM calls are routed through named lanes, not directly to models:

Lane	Purpose	Characteristics
`reasoning`	Complex analysis, multi-step planning	Highest quality, highest cost
`middle`	Balanced routing, general specialist work	Good quality, moderate cost
`triage`	Quick decisions, classification	Fast, low cost
`air_gap`	Offline / restricted environments	Local models only

Agents declare which lane they use in their YAML configuration. Individual agents can override the lane's default model.

7.2 Provider Fallback

The LLM router supports multiple providers (Anthropic, OpenAI, Mistral, local models). Only providers with configured API keys are included in the fallback chain. If a provider's key is missing, it is silently excluded rather than causing runtime errors.

7.3 Token Estimation

Before each LLM call, tokens are estimated to:

- Enforce budget limits before spending money
- Select the appropriate context window size
- Log accurate cost estimates

Token counting uses provider-specific methods when available, with fallback heuristics for unsupported providers.

8. Tool System

8.1 Tool Registry

The registry builds OpenAI function-calling tool definitions from two sources:

1. **Graph query templates.** Each template's parameters become tool arguments. The template's description and parameter types are used directly.
2. **SQL queries.** Hand-curated tool definitions map query names to parameter schemas with types, descriptions, defaults, and constraints.

8.2 Default Argument Injection

LLMs often pass placeholder values for parameters they cannot know (tenant ID, user ID). The registry detects these and auto-injects correct values:

```
# If LLM passes tenant_id="<UNKNOWN>" or tenant_id="null", replace with real value
if value in ("<UNKNOWN>", "unknown", "null", ""):
    value = default_args[key]
```

8.3 Parameter Clamping

Numeric parameters with `maximum` constraints are clamped server-side. If the LLM passes `limit=999999`, the system silently reduces it to the configured maximum. This prevents resource exhaustion from LLM overestimation.

9. Configuration

9.1 YAML-Driven Agent Definitions

Every agent is configured via YAML. No hardcoded behavior.

```
name: specialist_example
description: "Description for the supervisor's tool definition"

llm:
  lane: middle
  temperature: 0.2
  prompt_templates_dir: prompts/agents/example

graph:
  allowed_query_templates:
    - template_a
    - template_b

tool_calling:
  enabled: true
  max_tool_iterations: 5
  max_llm_calls: 8
  max_cost_usd: 0.15
  enabled_tools:
    - template_a
    - template_b
    - sql_query_x

database:
  allowed_queries:
    - sql_query_x
    - sql_query_y
```

9.2 Prompt Templates

Jinja2 templates with injected runtime variables:

```
You are an AI assistant with access to the following tools:
{{ available_tools }}

## Tool Selection Guide
For listing entities: Call fetch_entity_list
For entity details: Call fetch_entity_detail
...

## Rules
- Always pass tenant_id="{{ tenant_id }}" to database tools
- Never include raw queries in your response
- Never use emojis
```

The prompt template is the primary interface between the framework and domain-specific behavior. Changing agent behavior starts here.

9.3 Environment Variables

Configuration that varies between environments (dev, staging, prod) is managed through environment variables, not code. API keys, database URLs, feature flags, and service endpoints are all externalized.

10. Observability

10.1 Structured Logging

Every agent runtime call produces a structured log trail:

```
[agent_name] START query="user question" tools=12
[agent_name] iter=0 tool_choice=required
[agent_name] iter=0 LLM called tools: ['tool_a', 'tool_b']
[agent_name] TOOL_CALL tool_a args={"param": "value"}
[agent_name] TOOL_OK tool_a records=5 size=1842
[agent_name] FINAL_LLM raw_length=2340
[agent_name] PARSED responseType=entity_list blocks=3 payload=present
[agent_name] DONE status=success tools=[...] tokens=8200 cost=$0.03 duration=16300ms
```

This trail is sufficient to debug any agent interaction without reproducing the request.

10.2 Response Metadata

Every response includes non-rendered metadata:

```
{
  "meta": {
    "requestId": "unique-id",
    "toolsUsed": ["tool_a", "tool_b"],
    "iterations": 3,
    "tokensUsed": 8200,
    "costUsd": 0.03,
    "durationMs": 16300,
    "modelUsed": "middle"
  }
}
```

This enables cost tracking, performance monitoring, and debugging without additional instrumentation.

10.3 LLM Tracing (Optional)

Integration with LLM observability platforms (e.g., Langfuse) is built in but requires API keys to be configured. Once enabled, every LLM call is traced with full message history, tool calls, token counts, and costs.

11. Security Model

11.1 Trust Boundaries

```
Untrusted    | LLM output, user input
Semi-trusted | Tool results (from our databases, but shaped by LLM tool calls)
Trusted      | Configuration (YAML), environment variables, database schemas
```

LLM output is never trusted. It is always:

- Parsed through Pydantic models (structural validation)
- Filtered through output guardrails (content sanitization)
- Constrained by tool allowlists (capability limiting)

11.2 Defense in Depth

Layer	Protection
Input	Unicode normalization, injection pattern detection, length limits
Routing	Supervisor uses low temperature, tool-calling (not free text)
Tools	Double-gate allowlists, parameterized queries, parameter clamping
Output	Query stripping, PII redaction, size limits, depth limits
Budget	Per-request token caps, per-agent cost caps, per-node cost caps
Circuit	Specialist failures trigger cooldown, not infinite retry

11.3 Checklist for Every New Feature

- All database queries go through a safe executor with an allowlist
- Agent YAML lists only the queries it needs (least privilege)
- Tenant ID flows from the auth header, not from user input or LLM output
- LLM output is sanitized before reaching the frontend
- Budget limits are set in configuration (not unlimited by default)
- Error messages to the user are generic (no stack traces, no schema details)
- New endpoints validate authentication headers
- Numeric tool parameters have maximum constraints

12. Error Handling Contract

12.1 Error Categories

Category	Response	Logging	Recovery
Validation error	400 with field-level errors	Debug	User fixes input
Auth missing	401	Warn	User authenticates
Tenant mismatch	404 (not 403)	Warn	None (prevents info leak)

Category	Response	Logging	Recovery
Budget exceeded	Partial response with available data	Info	User starts new conversation
Tool failure	LLM receives error, adapts response	Warn	Automatic (LLM works around it)
LLM failure	Generic error message to user	Error	User retries
Database down	503	Error + alert	Ops intervention

12.2 Error Response Format

```
{
  "status": "error",
  "blocks": [
    {
      "type": "error",
      "content": {
        "severity": "error",
        "title": "Something went wrong",
        "body": "We could not process your request. Please try again.",
        "recoveryHint": "If this persists, try a simpler question."
      }
    }
  ]
}
```

Internal details (exception class, query text, stack trace) are logged server-side but never exposed to the client.

13. Adding Features: Step by Step

New Specialist Agent

1. Create YAML configuration file
2. Create prompt template (Jinja2)
3. (Optional) Create custom Python class if generic ReAct is insufficient
4. Restart. The registry discovers it automatically.
5. No changes to the supervisor, routing, or API layer.

New Database Query for an Agent

1. Define the query as a named, parameterized object in the query registry
2. Create a tool definition (description, parameter schema with types and constraints)
3. Add the query name to the agent's YAML allowlist
4. Add the tool name to the agent's YAML enabled tools list

New Graph Query Template

1. Create the parameterized template file
2. Create the metadata file (cost class, parameters, allowed agents)
3. Add the template name to the agent's YAML allowlist and enabled tools

New API Endpoint

1. Add the route to the API application
2. Apply input guardrails if it accepts user text
3. Apply output guardrails if it returns LLM-generated content
4. Add the path to the ingress configuration
5. Add authentication header validation

14. Anti-Patterns

Things we do not do, and why.

Anti-Pattern	Why Not	What We Do Instead
Let LLMs write SQL / Cypher	Injection risk, unpredictable cost	Pre-defined parameterized queries with allowlists
Trust LLM self-classification	Unreliable, inconsistent	Deterministic mapping from tool usage
Swallow exceptions	Hides failures, makes debugging impossible	Log everything, surface errors to users
Use emojis in responses	Unprofessional for enterprise, inconsistent rendering	Text-only responses, no exceptions
Hardcode secrets	Security risk, environment coupling	Environment variables only
Skip input validation on internal APIs	Internal does not mean trusted	Every endpoint validates
Create agents without budgets	Unbounded cost risk	Every agent has token and cost caps
Retry failed specialists immediately	Amplifies failures, wastes budget	Circuit breaker with exponential backoff
Pass tenant ID in request body	Spoofing risk	Derive from authenticated header only
Embed large schemas in every prompt	Token waste, cache pollution	Trim prompts, use deterministic post-processing

15. Key Abstractions

Abstraction	File	Responsibility
SupervisorAgent	`agents/supervisor.py`	Intent classification, specialist routing, circuit breaker
AgentRuntime	`agents/runtime.py`	ReAct loop, tool execution, structured output parsing
AgentRegistry	`agents/registry.py`	YAML discovery, specialist instantiation
ToolRegistry	`conversation/tools/registry.py`	Tool definition building, execution, default injection
SafeGraphExecutor	`graph_client/safe_executor.py`	Double-gate allowlist, parameterized template execution
SafeQueryExecutor	`database_client/safe_executor.py`	Named query enforcement, parameter clamping
StructuredResponse	`agents/response.py`	Canonical response envelope (blocks, payload, metadata)
InputGuardrails	`agents/guardrails.py`	Injection detection, normalization, length enforcement
OutputGuardrails	`agents/guardrails.py`	Query stripping, PII redaction, size enforcement
SSEStreamer	`streaming/sse.py`	Redis pub/sub for real-time event delivery
StepEmitter	`streaming/step_emitter.py`	Progress events during agent reasoning
LLMRouter	`llm/router.py`	Lane-based model selection, provider fallback
BudgetTracker	`schemas/internal.py`	Per-request and per-node cost tracking
AgentDefinition	`config/models.py`	YAML schema validation (Pydantic)

16. Performance Characteristics

Operation	Typical Latency	Bottleneck
Supervisor routing	1–2s	Single LLM call
Specialist (2–3 tool calls)	8–15s	LLM calls + database queries
Specialist (5+ tool calls)	15–25s	LLM calls (sequential)
Workflow pipeline (full)	60–120s	Plan generation LLM call
Database query (graph)	50–200ms	Neo4j traversal depth
Database query (SQL)	5–50ms	Query complexity
SSE event delivery	<10ms	Redis pub/sub
Session restore from cache	<5ms	Redis GET
Session restore from DB	50–100ms	Postgres query

The dominant cost in every request is LLM latency. Database queries are fast. Optimizing LLM calls (fewer iterations, smaller context windows, appropriate model selection via lanes) has the highest ROI.

This document describes the system as built. If the code disagrees with this document, update the document.